

摘要

掇蛋作为一种广受欢迎的四人制扑克游戏，因其复杂的规则、不完全信息以及对团队协作的极高要求，对人工智能（AI）构成了独特的挑战。本文旨在利用深度强化学习技术，构建一个具备较高水平博弈能力的 AI 掇蛋智能体。为实现此目标，我们首先搭建了一个支持完整对局逻辑的 Python 虚拟掇蛋环境。在此基础上，本文采用演员-评论家（Actor-Critic）框架作为核心算法，通过与特定策略进行对局的方式进行模型训练。为了有效处理掇蛋巨大的决策空间，我们设计了创新的状态与动作表示方法：将游戏状态编码为一个包含手牌、历史出牌及局势判断的 3049 维向量，并将复杂的出牌行为抽象为 375 种结构化动作，极大地降低了模型的学习难度。实验中，我们分别基于全连接网络和 Transformer 架构构建了 AI 智能体，并与随机策略的机器人进行对战测试。结果表明，本文训练的 AI 模型展现出强大的策略能力和团队协作意识，其中表现最优的 Transformer 模型在团队测试中胜率超过了 80%，并能频繁取得一、二名的成绩。本研究成功验证了深度强化学习在解决复杂、非完美信息、多智能体协作博弈问题上的有效性，为 AI 在棋牌游戏领域的发展提供了有价值的参考。

关键词：深度强化学习；掇蛋；演员-评论家算法

目录

1. 绪论.....	1
1.1. 背景.....	1
1.2. 研究现状.....	1
1.3. 文献综述.....	2
1.3.1. 深度 Q 网络.....	2
1.3.2. 演员评论家算法.....	2
1.3.3. 深度确定性策略梯度.....	3
1.3.4. PPO、DQN 在博弈中的表现.....	4
1.4. 本文结构.....	4
2. 方法与资源.....	5
2.1. 强化学习原理.....	5
2.2. 贝尔曼方程.....	6
2.3. 时序差分方法（Temporal Difference, TD）.....	7
2.4. 策略梯度方法（Policy Gradient）与 Actor-Critic 方法.....	8
2.5. 模型构建——深度强化学习.....	10
2.6. 数据来源及神经网络的构建.....	11
2.6.1. 状态输入设计（State Input）.....	12
2.6.2. 动作输出空间（Action Space）.....	12
2.6.3. Actor 网络结构.....	13
2.6.4. Critic 网络结构.....	15
2.6.5. 训练配置.....	16
2.6.6. 策略选择.....	16
3. 技术方案.....	16
3.1. 项目整体设计.....	16
3.2. 各模块技术细节.....	17

3.2.1. 规则逻辑层.....	17
3.2.2. AI 策略层（Actor-Critic）	18
3.2.3. 对局控制与训练流程.....	21
4. 效果验证.....	23
4.1. 虚拟对局测试方法说明.....	23
4.2. 单人测试结果.....	24
4.3. 团队测试结果.....	25
4.4. 胜率、排名指标图像.....	26
4.5. 训练效果总结.....	27
5. 总结与展望.....	28
5.1. 工作总结.....	28
5.2. 问题与不足.....	29
5.3. 未来展望.....	29
参考文献.....	30
附录.....	32

1. 绪论

1.1. 背景

掇蛋是一种非常受欢迎的扑克牌游戏，它主要流行于中国的江苏、安徽等地区。这个游戏不仅具有丰富的策略性，而且充满了娱乐性，深受广大玩家的喜爱。随着人工智能（AI）技术的迅猛发展，越来越多的研究者开始探索如何利用 AI 技术来实现复杂游戏的自动化。由于掇蛋游戏规则的复杂性以及对策略的高要求，它已经成为 AI 领域研究的一个非常有趣的课题。通过研究和开发能够玩掇蛋游戏的 AI 系统，不仅可以推动人工智能在游戏领域的应用，还能为研究人类策略行为、决策过程以及人机交互提供新的视角和数据。这不仅有助于提升 AI 技术在实际应用中的能力，同时也为理解人类在复杂决策过程中的行为模式提供了新的研究途径。

为了实现该课题的研究目标，本小组决定分两个研究方向，分别为：Multi-Agent(多智能体协作)和深度强化学习。

1.2. 研究现状

随着深度神经网络的发展，传统的强化学习方法被广泛扩展为深度强化学习（Deep Reinforcement Learning,DRL），使得智能体能够在高维、复杂的环境中学习近似最优策略。深度强化学习通过使用深度神经网络近似状态值函数、动作值函数或策略函数，使得模型可以直接从原始状态（如图像、向量等）中提取特征并做出决策。

DRL 技术已在多种博弈类任务中取得显著成果，如：

- **Atari 游戏：**Deep Q Network (DQN) 在无需先验知识的情况下学习到人类水平的策略；
- **围棋：**AlphaGo 利用深度神经网络与蒙特卡洛树搜索 (MCTS) 相结合，首次击败人类世界冠军；
- **多人对战游戏：**如 Dota 2 中的 OpenAI Five 使用 Proximal Policy Optimization (PPO) 训练多智能体协作对抗。

在博弈类问题中，深度强化学习的关键优势在于能够处理不确定性、动态环境和部分可观测信息，同时具备从历史经验中归纳策略的能力。

1.3. 文献综述

1.3.1. 深度 Q 网络

深度 Q 网络(Deep-Q-Network,DQN)是由 Mnih 等^[9]提出的。它依托强化学习中经典的 Q 学习,用一个深度网络近似价值函数为深度网络提供目标值,不断更新网络直至收敛。其中涉及 3 项关键技术: 1) 用经验重放技术打破了样本间的关联性,将采集到的样本先放入样本池,然后从池中随机选出一个样本用于网络训练; 2) 设置了目标网络来单独处理时间差分算法中的 TD 偏差,使训练的稳定性和收敛性得到极大的提高; 3) 利用卷积神经网络逼近行为值函数.其他学者也围绕 DQN 做了许多研究和改进。Hasselt 等^[10]提出了双 DQN 算法,使得值估计过于乐观这一问题得到解决。Schaul 等^[11]使用了经验优先回放技术,对经验的优先次序进行处理.此外,Osband 等^[12],Munos 等^[13],Francois-lavet^[14]等也分别从其他角度对 DQN 提出了改进。

1.3.2. 演员评论家算法

演员评论家算法(Actor-Critic,AC)的架构可以追溯到 30-40 年前。最早,Witten

于 1977 年提出了类似 AC 算法的方法;然后,Barto,Sutton 和 Anderson 等于 1983 年引入了 AC 架构。但是,AC 算法的研究难度和一些历史偶然因素使得之后学术界开始将研究重点转向基于价值的方法。之后的一段时间里,基于价值的方法和基于策略的方法都有了蓬勃的发展。AC 算法结合了两者的发展红利,在理论和实践方面再次有了长足的发展。

该结构包含两个网络:一个策略网络(Actor)和一个价值网络(Critic)。策略网络输出动作,价值网络评判动作。策略网络通过梯度计算公式进行更新,而价值网络根据目标值进行更新。相比以值函数为中心的算法,AC 算法应用了策略梯度的方法,这使得它能在连续动作或更高维动作空间中选取合适的动作;而 Q 学习难以实现这个目标,甚至会瘫痪。相比单纯策略梯度,AC 算法应用了 Q 学习或其他策略评估方法,使得 AC 算法能进行单步更新而不是回合更新,比单纯的策略梯度的效率更高。

1.3.3. 深度确定性策略梯度

深度 Q 网络是一种基于价值函数的方法,难以应对大的连续动作空间,无法输出离散状态动作值。Lillicrap 等^[15]于 2015 年提出的深度确定性策略梯度(Deep Deterministic Policy Gradients,DDPG)是基于上述 AC 算法的,结合确定性策略梯度算法(Deterministic Policy Gradients,DPG),在动作输出方面采用一个网络来拟合策略函数,直接输出动作,可以应对连续动作的输出以及更大的动作空间.此外,AC 模型还衍生出很多种算法,如异步优势行动评论家算法(A3C)^[16]和分布式近似策略优化算法(DPPO)^{[17][18]}.A 3 C 算法由 Mnih 于 2016 年提出,在 DQN 中为了破坏训练样本之间的相关性,采用了经验重放技术,即把训练样本缓存起来,每次训练时从中随机抽取一个 minibatch。在 A3C 中,利用多线程并行地去采集数据,每个线程以一个独立的智能体形式去搜索独立环境;同时,每个智能体还可以平行地利用不同的探索策略进行采样,这样每个线程得到的样本天然不相关,而且采样速度也更快。

1.3.4. PPO、DQN 在博弈中的表现

DQN (Deep Q-Network)

DQN 是深度强化学习的经典方法之一，其核心思想是使用深度神经网络拟合 Q 函数，即 $Q(s, a) \approx Q^*(s, a)$ ，通过贝尔曼方程进行更新。DQN 在博弈类问题中的优点在于：

- 能较快学习出动作价值函数，在有限状态空间下具有较强表现；
- 在如 Blackjack、Atari 等完全信息环境中效果显著。

但 DQN 在处理大动作空间、部分可观测信息（如扑克游戏）和策略不稳定的博弈时容易出现训练震荡，难以扩展到复杂策略场景。

PPO (Proximal Policy Optimization)

PPO 是基于策略梯度方法的优化算法，属于 Actor-Critic 框架的一种，核心优势在于：

- 使用“剪切损失函数”限制策略更新幅度，防止训练过程中策略突然崩坏；
- 支持连续动作空间和高维状态输入；
- 易于并行、自博弈训练，训练过程更加稳定。

在博弈类任务中，如 AlphaStar（星际争霸）、OpenAI Five（Dota2）、Leela Chess Zero（国际象棋）等，PPO 被广泛用于训练高性能智能体，特别适合多智能体对抗或协作情境。

相比 DQN，PPO 更适合本项目中掼蛋这种具有大量合法动作结构、出牌花色组合、非完全信息博弈特征的环境。

1.4. 本文结构

本文主要围绕掼蛋 AI 系统的设计与实现展开，内容安排如下：

第 1 章 绪论：介绍本项目的研究背景和意义，概述当前多智能体协作与深度强化学习领域的研究现状与发展趋势，梳理相关文献，为本项目的研究提供理论基础。

第 2 章 方法与资源：详细阐述本项目所采用的主要技术方法，包括多智能体协作原理、深度强化学习算法，特别是 Actor-Critic 方法的应用。还包括神经网络结构设计、训练数据来源与预处理方法等资源支持。

第 3 章 技术方案：对整个掼蛋 AI 系统的架构进行分层讲解，分别说明规则逻辑模块、AI 策略模块、训练控制模块的设计与实现细节。

第 4 章 效果验证：通过一系列实验对本系统的功能正确性、策略有效性以及训练性能进行验证，包含训练前后策略的对比实验、胜率指标图示及团队测试结果等。

第 5 章 总结与展望：总结本项目的工作成果与创新点，分析目前存在的问题与不足，并展望未来可能的改进方向与研究扩展路径。

2. 方法与资源

2.1. 强化学习原理

强化学习（reinforcement learning, RL）讨论的问题是智能体（agent）怎么在复杂、不确定的环境（environment）里面去最大化它能获得的奖励。如图 1 所示，强化学习由两部分组成：智能体和环境。

在强化学习过程中，智能体与环境一直在交互。智能体在环境里面获取某个状态后，它会利用该状态输出一个动作（action），这个动作也称为决策（decision）。然后这个动作会在环境之中被执行，环境会根据智能体采取的动作，输出下一个状态以及当前这个动作带来的奖励。智能体的目的就是尽可能多地从环境中获取奖励。

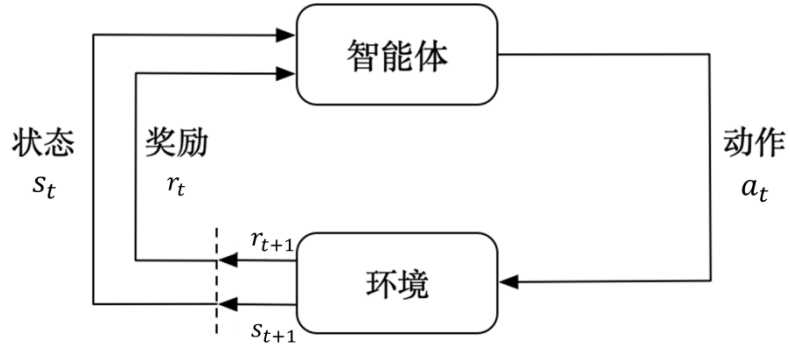


图 1 智能体环境的交互

在强化学习中，智能体与环境就是这样进行交互的，这个交互过程可以通过马尔可夫决策过程来表示：

$$(S, A, P, R, \gamma)$$

其中， S 表示状态空间， A 表示智能体执行的动作空间集合， P 表示执行动作的概率， R 表示奖励， γ 表示衰减因子。

2.2. 贝尔曼方程

贝尔曼方程由理查德·贝尔曼提出，是动态规划和强化学习中的核心方程。它将一个状态（或状态-动作对）的价值与其后续状态（或状态-动作对）的价值联系起来，体现了价值函数之间的迭代关系。简单来说，通过求解贝尔曼方程，可以在理论上得到所有状态下的价值，这样就有助于衡量每一个状态的“好坏”。

为了引入贝尔曼方程，我们首先需要定义每一个状态下的价值：

$$V^\pi(s) = E_\pi[G_t | S_t = s]$$

其中 G_t 是累积回报函数，通常是折扣回报，定义如下：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

γ 是折扣因子（Discount Factor， $0 \leq \gamma \leq 1$ ），用于平衡即时奖励和未来奖励的

重要性。如果 γ 接近 0，智能体更关注即时奖励（短视）；如果 γ 接近 1，智能体更关注长期累积奖励（远视）。

为了将其应用到马尔可夫决策过程中，还需定义动作价值函数：

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

相比状态价值函数，动作价值函数更进一步地，体现了在某个状态下采取特定动作的价值大小。

在定义状态价值函数和动作价值函数之后，根据全期望公式，可以得到期望形式的贝尔曼方程：

$$V^\pi(s) = \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s]$$

同理，对动作价值函数：

$$Q^\pi(s, a) = \mathbb{E}[r_{t+1} + \gamma Q^\pi(s_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

两者具有紧密的联系：

$$V^\pi(s) = \mathbb{E}_a[Q^\pi(s, a)]$$

事实上，贝尔曼方程就是当前状态与未来状态的迭代关系，表示当前状态的价值函数可以通过下个状态的价值函数来计算，而大部分的强化学习的任务都是通过不断迭代贝尔曼方程，从而得到最优的结果。

2.3. 时序差分方法（Temporal Difference, TD）

为了得到贝尔曼方程的解，我们引入时序差分方法。时序差分方法是强化学习中一种核心的免模型（model-free）方法，这种方法无需构建学习任务的全局奖励函数。同时，也不需要知道环境的转移概率，与蒙特卡罗方法类似（Monte Carlo）。但是相比蒙特卡罗方法，时序差分方法不需要采集完整的轨迹（Episode）就可以进行强化学习的任务，具有高效、简便的特点。简单来说，时序差分学习是“从未来估计的未来”来学习。它用一个估计值去更新另一个估计值。

时序差分的更新过程如下：

$$V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

这个更新过程其实相当直观，考虑贝尔曼方程的形式可以很容易地看出，时序差分方法实则是构建了以下损失函数：

$$Loss = \|r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\|_2^2$$

表示在合适的即时奖励下，时序差分的目标则满足贝尔曼方程的解。在实际情况下， $V(s_t)$ 是任务中需要估计的值，而 $r_{t+1} + \gamma V(s_{t+1})$ 代表真实的状态价值函数，通常由估计值来近似。事实上，时序差分方法是一种随机梯度下降方法。

2.4. 策略梯度方法（Policy Gradient）与 Actor-Critic 方法

在强化学习中，策略梯度方法和 Actor-Critic 方法是两种非常重要的、用于解决控制问题的算法类别。它们的目标都是找到最优策略，使得智能体在环境中获得最大的累积奖励。

策略梯度方法直接学习一个参数化的策略 $\pi_\theta(a|s)$ ，其中 θ 是策略的参数。它们的目标是找到能够最大化期望回报的策略参数 θ 。与价值函数方法(如 Q-learning、SARSA 等)不同，策略梯度方法不直接学习价值函数，而是直接优化策略。

而策略梯度方法的思想也相当直观，那就是最大化策略的期望回报。在某一场游戏的某一个回合里面，我们会得到一系列奖励 $R(\tau)$ 。我们要做的就是调整策略内部的参数 θ ，使得 $R(\tau)$ 的值越大越好。但实际上 $R(\tau)$ 并不只是一个标量(scalar)，它是一个随机变量，因为策略在给定同样的状态下会采取什么样的动作，这是有随机性的。环境在给定同样的观测时要采取什么样的动作，要产生什么样的观测，本身也是有随机性的，所以 $R(\tau)$ 是一个随机变量。我们能够计算的是 $R(\tau)$ 的期望值。给定某一组参数 θ ，我们可计算 r_θ 的期望值为：

$$\bar{R}_\theta = \sum_{\tau} R(\tau) p_\theta(\tau)$$

而方法被称为策略梯度的原因就是，在以上表达式中，对等号两边分别关于

策略 θ 取梯度：

$$\nabla \bar{R}_\theta = \sum_{\tau} R(\tau) \nabla p_\theta(\tau)$$

但是为了执行随机梯度上升策略，需要对以上等式做一些变换：

$$\begin{aligned} \nabla \bar{R}_\theta &= \sum_{\tau} R(\tau) \nabla p_\theta(\tau) \\ &= \sum_{\tau} R(\tau) p_\theta(\tau) \frac{\nabla p_\theta(\tau)}{p_\theta(\tau)} \\ &= \sum_{\tau} R(\tau) p_\theta(\tau) \nabla \log p_\theta(\tau) \\ &= \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)] \end{aligned}$$

在实际操作时，该期望是难以计算的，所以常常使用样本均值来代替期望，即：

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)$$

再经过随机梯度上升，即可更新策略：

$$\theta \leftarrow \theta + \eta R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)$$

并且在实际应用当中，常常使用一种添加“基线”的技巧，这是为了解决奖励总是正值的情况，此时将奖励减去一个基准值 b ，即：

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} (R(\tau^n) - b) \nabla \log p_\theta(a_t^n | s_t^n)$$

这样做的好处不仅仅是可以平衡奖励大小，而且会减小估计的方差。 b 怎么设置呢？我们可以对 τ 的值取期望，计算 τ 的平均值，令 $b \approx E[R(\tau)]$ 。所以在训练的时候，我们会不断地把 $R(\tau)$ 的值记录下来，会不断地计算 $R(\tau)$ 的平均值，把这个平均值当作 b 来使用（在 A2C 方法中就会应用到）。

Actor-Critic 方法将策略梯度与时序差分结合，其中 Actor 表示智能体所执行的策略，而 Critic 表示状态价值函数 $V^\pi(s)$ 对当前策略的值函数进行估计，即评估演员的好坏。借助于价值函数，演员-评论员算法可以进行单步参数更新，不需

要等到回合结束才进行更新。在演员-评论员算法里面，最知名的算法就是异步优势演员-评论员算法。如果我们去掉异步，则为优势演员-评论员（advantage actor-critic, A2C）算法。

2.5. 模型构建——深度强化学习

在传统的强化学习当中，状态空间大多数是离散且低维的，所以对每个状态，可以制定特定的策略来进行学习。但是对于我们当前构建攒蛋智能体的任务来说，对每个输入状态来分配不同的策略是难以实现的，因为输入牌型不仅包含当前回合，也包括历史出牌，使得它们的组合无穷无尽。在这种条件下，就需要一个“巨大”的函数来逼近所有的策略，而神经网络恰好能够做到这一点，于是深度强化学习应运而生。

我们利用 A2C 方法来搭建整个深度强化学习的框架。整体算法流程如下：

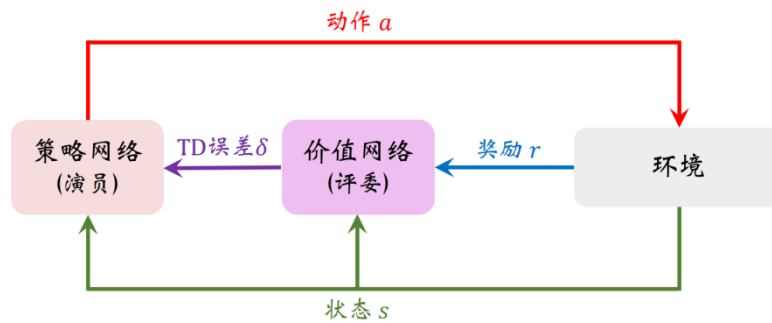


图 2A2C 算法流程

A2C(Advantage Actor-Critic)算法：

- 1) 初始化 Actor 网络 $\pi(a|s; \theta)$ 的参数 θ ，初始化 Critic 网络 $V(s; \phi)$ 的参数 ϕ 。
- 2) 设置学习率 α_θ 和 α_ϕ 。
- 3) 设置折扣因子 γ 。
- 4) 智能体在当前状态 $s_t^{(i)}$ 下，根据策略 $\pi(a|s_t^{(i)}; \theta)$ 采样一个动作 $a_t^{(i)}$ 。
- 5) 执行动作 $a_t^{(i)}$ ，观察奖励 $r_t^{(i)}$ 和下一个状态 $s_{t+1}^{(i)}$ 。

- 6) 将 $(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)})$ 输入到 Critic 网络, 用时序差分方法进行更新: $\phi_{t+1} \leftarrow \phi_t + \alpha_\phi (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)) \nabla_\phi V_\phi(s_t)$, 其中, $(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))$ 被称为优势函数。
- 7) 将优势函数的值传回到 Actor 网络, 用策略梯度方法进行更新:

$$\theta \leftarrow \theta + \alpha_\theta (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)) \nabla_\theta \log \pi_\theta(a_t^n | s_t^n)。$$
- 8) 重复以上流程, 直到达到最大迭代步数或收敛。

在实际应用当中, A2C 方法会存在一定的问题, 这是因为所有的对局数据都由一个 Actor 网络生成, 导致数据具有较高的相关性。而异步优势演员-评论员 (asynchronous advantage actor-critic, A3C) 则可以缓解这个问题。简单来说, A3C 是 A2C 的基础上增加了更多的 Actor 网络, 每个 Actor 在相互独立地与其本地环境交互, 使用平行探索的方法将梯度信息传回全局的 Actor-Critic 网络。而为了提高效率, 只有全局的网络会执行更新操作, 而本地的网络则定期拉取全局网络的参数即可。在掼蛋智能体的实验当中, 我们也分别训练了 A2C 模型与 A3C, 进行对比。

2.6. 数据来源及神经网络的构建

我们使用 python 搭建了一个掼蛋虚拟环境, 以自对弈的形式进行强化学习。

在初期的训练中, 为了节约时间用于前后端的工作, 我们采用只包含全连接层的神经网络来进行训练, 而在后续的训练中, 我们更换了 transformer 来进行训练。

本项目采用深度强化学习中的 Actor-Critic 架构, 构建了两个独立的神经网络模型: Actor 网络用于策略生成, Critic 网络用于状态价值估计。训练数据主要来源于模型与随机策略自博弈生成的对局记录。

2.6.1. 状态输入设计 (State Input)

每一轮出牌中，当前玩家的观察状态编码为一个 3049 维的向量，详细描述见下表：

表格 1 输入状态独热编码

特征	维度	描述
当前玩家的手牌	108	两副牌的 54 张牌的是否在手信息
其他玩家手牌数量	3	剩余牌数（归一化）
每个玩家最近动作	108×4	上轮每位玩家的出牌
出过的牌	108×3	记录其他玩家已经打出的牌
当前级牌	13	One-hot 编码
最近 20 次动作	$108 \times 4 \times 5$	5 轮历史，每轮 4 玩家，每人 108 维
协作状态	3	与队友协作的程度
压制状态	3	对敌方的打击程度
辅助状态	3	是否有意铺路（辅助队友）
总维度	3049	完整输入维度

108 维具体为{黑桃、红桃、梅花、方块}与{2、3、4、...、K、A}作积，生成 52 维。具体是{黑桃、红桃、梅花、方块}2、{黑桃、红桃、梅花、方块}3、...，如此再重复一遍为 104 为，再加上{小王、大王、小王、大王}得到最终 108 维。

手牌数量归一化方法为 $\text{MIN}\{\text{手牌数}, 26\} / 26$ ，状态标记均用 3 维独热编码分别表示不能、选择、拒绝。

2.6.2. 动作输出空间 (Action Space)

本项目将所有可能的“出牌结构”抽象为动作全集 M ，由外部文件 doudizhu_actions.json 统一编号，共包含 375 种结构动作。每个结构动作如“单张 5”、“三带二”、“四炸”等，真实出牌时需从合法的花色组合中选择。下表为一些

数据示例：

表格 2 动作空间示例

type	points	logic_point	id
None	[]	0	0
single	[2]	2	1
pair	[2,2]	2	18
triple	[2,2,2]	2	35
4_bomb	[2,2,2,2]	2	49
5_bomb	[2,2,2,2,2]	2	63
6_bomb	[2,2,2,2,2,2]	2	77
7_bomb	[2,2,2,2,2,2,2]	2	91
8_bomb	[2,2,2,2,2,2,2,2]	2	105
joker_bomb	[16,16,17,17]	1	119
three_with_pair	[2,2,2,3,3]	2	120
straight	[2,3,4,5,14]	1	330
pair_chain	[2,2,3,3,14,14]	1	340
gangban	[2,2,2,14,14,14]	1	352
flush_rocket	[2,3,4,5,14]	1	365

实际上 M 是一个 JSON 文件，有些记录不只表中的 4 种字段，还包括一些其他信息，留待后续扩展，但本项目暂时还未用到这些字段。Type 代表牌型，可以加速牌力判断；Points 固定了牌的排列方式，简化了动作空间；Logic_point 用于比较牌力；Id 是网络选择动作的输出索引。

2.6.3. Actor 网络结构

全连接网络结构如下：

Actor 网络接收状态向量作为输入，输出一个维度为 375 的概率分布，表示每个结构动作的选择概率。网络结构如下：

输入层：3049 维状态向量

隐藏层 1：全连接层（512 维），ReLU 激活

隐藏层 2：全连接层（512 维），ReLU 激活

输出层：全连接层（375 维），Softmax 输出概率分布

在推理阶段，还会加上掩码 mask 限制非法动作的概率为零（通过对 logits 加

上极小值实现）。

Transformer 网络结构如下：

输入层： 输入一个 3049 维的状态向量，该向量编码了当前游戏状态的所有相关信息（例如手牌、历史出牌、牌桌状态等）。

输入投影层： 将 3049 维的输入状态通过一个全连接层投影到一个更高维的隐藏空间（512 维），以便 Transformer 编码器进行处理。

层类型： 全连接层 (nn.Linear)

输出维度： 512

Transformer 编码器： 这是一个核心模块，用于捕获状态特征之间的复杂关系和上下文信息。它由 1 层 Transformer 编码器层组成。

层类型： nn.TransformerEncoderLayer

模型维度： 512

头数： 8

前馈网络维度： $512 \times 4 = 2048$

Dropout： 0.1，

层数： 1

输出网络： 经过 Transformer 编码器处理后的特征向量通过一个序贯网络，最终输出动作的概率分布。

隐藏层 1： 全连接层（256 维），后接 ReLU 激活函数。

隐藏层 2： 全连接层（输出维度等于总动作空间维度 `action_dim`），不带激活函数，其输出为 Logits。

激活函数： 最终在输出层应用 Softmax 激活函数，将 Logits 转换为所有可能动作的概率分布。网络还支持通过温度参数 (temperature) 调整 Softmax 的锐度，从而控制探索程度。在训练过程中，温度会动态调整，从 1.0 逐渐降低到 0.1，以在初期鼓励探索，后期趋向确定性选择。

动作掩码： 在前向传播过程中，网络接受一个布尔类型的动作掩码。此掩码用于将无效动作对应的 Logits 设置为一个非常大的负值 ($-1e9$)，确保这些动作在 Softmax 后具有接近零的概率，从而防止代理选择非法动作。

2.6.4. Critic 网络结构

全连接网络结构如下：

Critic 网络用于估计当前状态的状态值 $V(s)$ ，采用类似结构但输出为标量：

输入层：3049 维状态向量

隐藏层 1：全连接层（256 维），ReLU 激活

隐藏层 2：全连接层（256 维），ReLU 激活

输出层：1 维状态值估计

Transformer 网络结构如下：

输入层：输入一个 3049 维的状态向量，与 Actor 网络的输入相同。

输入投影层：将 3049 维的输入状态通过一个全连接层投影到隐藏空间（512 维）。

层类型：全连接层 (`nn.Linear`)

输出维度：512

Transformer 编码器：与 Actor 网络结构相同，用于从状态中提取高级特征。

层类型：`nn.TransformerEncoderLayer`

模型维度：512

头数：8

前馈网络维度： $512 \times 4 = 2048$

Dropout：0.1

层数：1

输出网络：经过 Transformer 编码器处理后的特征向量通过一个序贯网络，最终输出一个标量状态值。

隐藏层 1：全连接层（256 维），后接 ReLU 激活函数。

隐藏层 2：全连接层（128 维），后接 ReLU 激活函数。

输出层：全连接层（1 维），输出当前状态的估计值。

激活函数：隐藏层使用 ReLU 激活函数。最终输出层不使用激活函数，直接输出状态值。

2.6.5. 训练配置

优化器: Adam

学习率: $1e-4$

折扣因子 $\gamma = 0.9$

每一局游戏结束后, 根据 TD(0)更新 Critic, 使用 Advantage 对 Actor 进行策略梯度更新。

2.6.6. 策略选择

Actor 网络在推理阶段根据概率分布进行采样 (使用 `torch.multinomial`), 然后枚举所有符合结构的花色组合 `enumerate_colorful_actions(...)`, 从中随机选一个出牌组合作为最终动作。

3. 技术方案

3.1. 项目整体设计

为了进行深度强化学习, 我们先使用 Python 搭建了一个虚拟掇蛋环境 (GuandanGame), 该环境满足支持运行一个基础掇蛋游戏的要求, 同时为了加快训练, 将部分函数使用 Cython 编译。我们采用优势演员-评论家 (A2C) 算法, 并设计了一系列长期、即时奖励以及惩罚。

掇蛋的动作空间介于离散与连续之间: 如果计算花色以及牌之间的排列, 动作空间显然比一般的离散动作空间大且存在冗余, 但又不适用于连续的动作空间。因此我们对出牌按结构进行编码并不考虑花色, 将动作空间压缩在了 375 维。

系统架构总览：

- 1) 规则逻辑层（GuandanGame、Rules）
- 2) AI 策略层（Actor-Critic）
- 3) 训练与对局控制层（actor.py、FrontendGame.py）

3.2. 各模块技术细节

3.2.1. 规则逻辑层

一) 规则设计

我们的基础掇蛋游戏包括：发牌、判断出牌合法性、维护掇蛋游戏、推进游戏运行、选择座位、接风等、A 可当 1 参与组牌。

该环境不支持的掇蛋特殊规则有：

逢人配：红桃级牌可以作为万能牌，充当除王牌外的任意牌参与组牌，当红桃级牌单独打出视作普通级牌。由于会导致智能体的动作空间过于大，不利于训练，因此放弃该规则。

贡牌¹：进贡：在每局游戏开始前，上一局的下游玩家需向上游玩家进贡一张手中最大的牌，接受进贡的玩家需还一张不大于 10 的牌给进贡者；还贡：还贡的牌必须是自己手中最大的牌，且不能超过进贡牌的价值。如果还贡方手中没有最大牌，可以选择不还贡；抗贡：如果进贡的玩家手中持有两张大王，可以拒绝进贡任何牌给赢家，这一过程称为“抗贡”。由于该规则与本项目主要训练目标无关，因此放弃该规则。

升级级牌：两队级牌独立，均从 2 开始，升到 A 为止。获胜方根据得分升 1-3 级不等，下一局的级牌为上一局胜方升级后的级牌。当一队级牌升级到 A 后，必须要再获得一次非一四名的胜利才能取得掇蛋的胜利。由于该规则实际上与训练的过程无关，智能体只需要在完整的小局切片中采样即可训练，因此在训练时并未采样该规则。但我们仍保留了该规则的代码，用于正式打牌时使用。

¹ 《江苏省掇蛋竞赛简易规则》（2017 版）制订出台 www.jssports.gov.cn

二) 发牌与牌面编码

我们创建了一个 `Player` 类，用于保存每个玩家的手牌、已经打出的牌和上一次出牌情况；在 `give_cards.py` 中创建了 2 组牌，打乱后分发给 4 位玩家。每张牌都拥有属于自己的索引，用于编制状态信息以及反映射。

三) 牌型定义与识别

我们首先在 `create_M.py` 中创建了所有出牌结构动作统一编码，即动作空间 (M)，保存在 `doudizhu_actions.json` 中，同时生成了逐行的非标准 JSON 文件 `actions.jsonl` 便于调试。

结构动作包括：`single`、`pair`、`triple`、`gangban`、`flush_rocket` 等

四) 出牌合法性与比较

如果两次出牌均是随机机器人给出的，我们首先对出牌编制索引，`key` 为无花色牌，值为牌数，例如 `{6:2}` 表示有 2 张 6。之后搜索对子、三张的数量等判断牌型，再根据牌型是否为炸弹先快速比较。如果是炸弹压非炸弹，或非炸弹压炸弹，直接得到合法与不合法的结构。对于均非炸弹的情况，如果牌型相同则进一步比点数，如果牌型不一样则不合法。对于均炸弹的情况，先比炸弹大小（天王炸 > 8-6 > 炸同花顺 > 5-4 炸），如同牌型再比较点数。

如果有一方出牌是训练模型给出的，由于自带 `type`、`logic_point` 等信息，此时对随机机器人的出牌通过 `enumerate_colorful_actions(a, hand, level_rank)` 函数编码成动作空间中的牌型。之后的比较与上方相同。如果两次出牌均是动作空间中的结构，与上方类似。

3.2.2. AI 策略层 (Actor-Critic)

3.2.2.1. 状态定义

我们使用 `_get_obs()` 方法得到状态编码，首先使用 `obs=np.zeros(3049)` 得到 3049 维空状态，然后从 `GuandanGame` 的 `players` 属性中获取它维护的 4 个 `Player`

类，得到当前玩家的手牌并使用 `encode_hand_108()` 将手牌编码，长 108，其他 3 人获取手牌数量后归一化。

之后同样从 `Player` 维护的 `last_played_cards` 属性获取 4 人上次出牌并编码，共 $108 \times 4 = 432$ 维。从 `played_cards` 属性获取另外 3 人已出的牌并编码，共 $108 \times 3 = 324$ 维。

再从 `GuandanGame` 的 `active_level` 属性中获取级牌并编码，13 维。最后再从 `history` 属性获取最近 20 次动作并编码，共 2160 维。

将上述向量全部相加后编码入 `obs`，并在返回 `obs` 前检测最终维数是否匹配。

3.2.2.2. 动作定义

我们根据不同牌型分别生成具体内容，直接使用列表表达式生成各牌型的具体内容，最后手动添加 `pass` 牌型并拼接，得到动作空间 `M` 并保存在 `doudizhu_actions.json` 中。详细信息见附录表格 5 完整的动作空间。

3.2.2.3. 策略模型

在本项目中，我们使用 `PyTorch` 框架分别构建了 `Actor` 网络和 `Critic` 网络，用于策略生成与状态价值评估。两者均为前馈神经网络，结构如下：

Actor 网络设计

`Actor` 网络用于从当前状态向量中输出动作结构的概率分布。其网络结构为三层全连接网络：

输入层： 维度为 3049，对应于状态编码向量；

隐藏层： 两层隐藏层，均为 512 维全连接层，激活函数使用 `ReLU`；

输出层： 维度为 `action_dim`（即所有结构动作的个数），无激活函数；

输出处理： 输出 `logits` 后，使用 `softmax` 函数得到动作概率分布，并引入掩码(`mask`)对非法动作概率进行惩罚，使其概率趋近于零。

在实现中，掩码机制如下所示：

```
logits = self.net(x)
logits = logits + (mask - 1) * 1e9 # 非法动作的 logits 极小化
```

```
return F.softmax(logits, dim=-1)
```

此机制确保了最终采样出的动作总是合法的结构动作。

Critic 网络设计

Critic 网络用于对当前状态 s 的状态值 $V(s)$ 进行评估，用于计算 Actor 的 Advantage 和 TD 误差。结构与 Actor 类似，但更轻量：

输入层：同样为 3049 维状态向量；

隐藏层：两层，每层为 256 维，ReLU 激活；

输出层：1 维的线性输出，对应状态值 $V(s)$ 。

Critic 网络的输出不使用激活函数，表示任意实数值：

```
self.net = nn.Sequential(  
    nn.Linear(state_dim, 256),  
    nn.ReLU(),  
    nn.Linear(256, 256),  
    nn.ReLU(),  
    nn.Linear(256, 1)  
)
```

该架构兼具表达能力与训练稳定性，能够有效配合 Actor 网络完成策略更新。

3.2.2.4. 强化学习算法

为实现策略和价值网络的联合训练，本项目采用了基于 Actor-Critic 架构的策略梯度方法，并在每局攒蛋对局结束后基于采样到的一批状态-动作-奖励序列执行一次批量训练更新。具体过程如下：

首先，将智能体在对局中经历的轨迹（即状态、动作、即时奖励）整理为一个 batch，并将其转换为 PyTorch 的张量表示。每条样本包含：

- 当前状态 s_t
- 执行动作 a_t
- 得到的即时奖励 r_t

为构造 TD（时序差分）学习目标，还需要为每个状态生成对应的下一状态

s_{t+1} 。在本项目中，使用状态序列在时间上平移构造 s_{t+1} ，对于最后一条样本，由于没有后继状态，则设置为全零状态。同时设定一个终止标志张量 $done$ ，指示最后一条为终止状态。

接着，Critic 网络对每个状态 s_t 和其后继状态 s_{t+1} 分别估计其状态值函数 $V(s_t)$ 和 $V(s_{t+1})$ ，以计算 TD Advantage：

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

该 Advantage 表示当前动作相对于 Critic 估值的优势程度，用于指导策略网络的优化。

Critic 更新

Critic 的损失函数为均方误差（MSE）形式的 TD 误差平方：

$$L_{critic} = \frac{1}{N} \sum_{t=1}^N (A_t)^2$$

通过最小化该损失，可以使 Critic 网络更准确地逼近状态值函数 $V(s)$ 。

Actor 更新

Actor 网络输出当前状态下所有动作的概率分布 $\pi(a_t|s_t)$ ，训练目标是最大化 Advantage 加权的对数概率期望，即：

$$L_{actor} = -\frac{1}{N} \sum_{t=1}^N \log \pi(a_t|s_t) \cdot A_t$$

在实际计算中，取负号以转化为最小化目标，且使用`detach()`操作避免 Advantage 在 Actor 反向传播中被更新。

最后，分别对 Actor 和 Critic 的参数进行梯度更新，优化器采用 Adam，学习率设置为 1×10^{-4} 。

3.2.3. 对局控制与训练流程

3.2.3.1. 对局流程

本项目的对局流程由 `GuandanGame` 类负责控制，完整模拟了真实掇蛋四人对局的逻辑。每局游戏中，四位玩家按顺序依次出牌，当前版本中第 0 号玩家由

Actor 网络控制，其余三位玩家使用随机策略（可作为 baseline 或干扰策略）。

- 每个出牌轮次中，系统将依据以下流程执行：
- 若当前为自由出牌轮（上一轮被重置），玩家可以主动选择任意合法牌型；
- 否则，玩家仅能选择能够“压过上家”出牌的组合；
- 出牌后更新当前状态，包括手牌、历史动作、出完牌玩家记录等；
- 若出现“全体 Pass”或某方出完牌等事件，将触发新一轮或结束游戏。

此外，系统还支持特殊掇蛋的接风特殊规则，确保游戏流程与现实一致。

3.2.3.2. 训练循环

训练过程由 actor.py 文件中的 run_training()函数实现，其核心逻辑包括对局生成、状态记录、奖励计算与网络更新。流程如下：

游戏初始化与对局生成

每一轮训练（即一局完整掇蛋）中，首先创建一个 GuandanGame 实例并开始自动对局。每轮玩家依序出牌，并根据当前轮次状态调用 Actor 网络或随机策略决策。

状态、动作与奖励记录

当第 0 号 AI 玩家出牌时，系统将：

- 获取状态向量（3049 维）；
- 根据出牌结构动作 ID 进行动作采样；
- 枚举所有合法出牌组合并随机选择具体牌面；
- 根据出牌类型与对局逻辑计算即时奖励（考虑牌型质量、炸弹拆分惩罚等）；
- 将三元组（状态、动作 ID、奖励）存入 memory 轨迹中。

训练数据处理与网络更新

在一局结束后，将整个 memory 中的 reward 进行带衰减因子的未来奖励加权叠加，形成完整的 R_t 序列。随后调用 train_on_batch()函数执行一次 Actor-Critic 联合训练。核心步骤包括：

- 使用 Critic 网络计算每个状态的状态值 $V(s)$ 和 TD Advantage；
- 使用 Advantage 对 Actor 网络进行策略梯度更新；

- 对 Critic 网络使用 TD 误差最小化进行回归更新。

模型保存与日志输出

每训练 200 局，将当前 Actor 和 Critic 模型权重保存至磁盘，供后续加载与推理使用；每 20 局输出一次损失与胜场日志，方便监控训练动态。

该训练流程结合了自博弈数据生成、蒙特卡洛奖惩修正、合法动作采样与策略梯度优化，实现了一个端到端的深度强化学习系统，适用于复杂不完全信息博弈场景。

4. 效果验证

4.1. 虚拟对局测试方法说明

计算训练的模型胜率可以与机器人或大量玩家对战。由于与大量真人对战的条件难以满足，且与市面上成熟的算法或神经网络对战缺少桥梁，我们仅与我们搭建的环境中的随机机器人进行对战以取得胜率指标。

虚拟对战有两种模式：单人与团队。下列数据均由经 200 场虚拟对局后计算得出，以排除发牌偶然性。

在单人测试中，一局游戏由 1 个模型和 3 个随机机器人参与，我们训练的模型和一个机器人组队，对战另外 2 个机器人的队伍。

在团队测试中，一局游戏由 2 个模型和 2 个随机机器人参与，一队由 2 个相同的模型组成，对战另外 2 个机器人的队伍。

一小局为一个 Episode，每个小局开始时拥有随机的级牌，测试的模型的出牌位置固定，在单人测试中为 1 号位（即第一个出牌的），在团队测试中为 1 号和 3 号位。

4.2. 单人测试结果

表格 3 所有模型测试结果汇总（单人）

Episode	胜率(%)	第一手出完(%)	一二名(%)	一三名(%)	一四名(%)
400	52.50	25.50	39.05	39.05	21.90
600	46.00	24.50	42.39	34.78	22.83
800	60.50	36.00	36.36	39.67	23.97
1000	65.00	35.50	39.23	43.85	16.92
1200	55.00	31.50	32.73	45.45	21.82
1400	64.00	42.00	32.81	45.31	21.88
1600	63.00	47.50	35.71	45.24	19.05
1800	66.00	40.00	31.82	44.70	23.48
2000	64.00	46.00	32.81	46.09	21.09
2200	68.50	53.50	36.50	47.45	16.06
2400	58.50	33.50	37.61	37.61	24.79
2600	71.50	53.00	37.06	42.66	20.28
2800	68.00	50.00	38.97	41.91	19.12
3000	62.00	40.00	33.06	42.74	24.19

胜率指的是训练模型所在队伍胜利的比例，该指标考察了模型的综合能力；第一手出完指的是在所有测试对局中训练的模型是第一个出完的(头游)的比例，该指标只在单人测试中考察，考察模型的单体素养；一二名指的是训练模型所在队伍在胜场中的排名分布，一三名、一四名以此类推，该指标考察了模型的团队协作能力，胜者队友的名次同样也是攒蛋中重要的指标，它决定了级牌升级的点数。

4.3. 团队测试结果

表格 4 所有模型测试结果汇总（团队）

Episode	胜率(%)	一二名(%)	一三名(%)	一四名(%)
400	52.50	37.14	40.00	22.86
600	55.50	27.93	59.46	12.61
800	61.50	31.71	47.97	20.33
1000	61.00	47.54	37.70	14.75
1200	65.50	48.09	35.88	16.03
1400	66.00	52.27	30.30	17.42
1600	73.50	55.78	27.21	17.01
1800	74.50	46.98	38.26	14.77
2000	76.50	49.67	35.95	14.38
2200	81.50	57.67	26.99	15.34
2400	70.50	52.48	31.21	16.31
2600	80.00	61.88	26.25	11.88
2800	84.00	64.88	24.40	10.71
3000	79.00	55.06	25.95	18.99

各指标详细描述见上方表注释

4.4. 胜率、排名指标图像

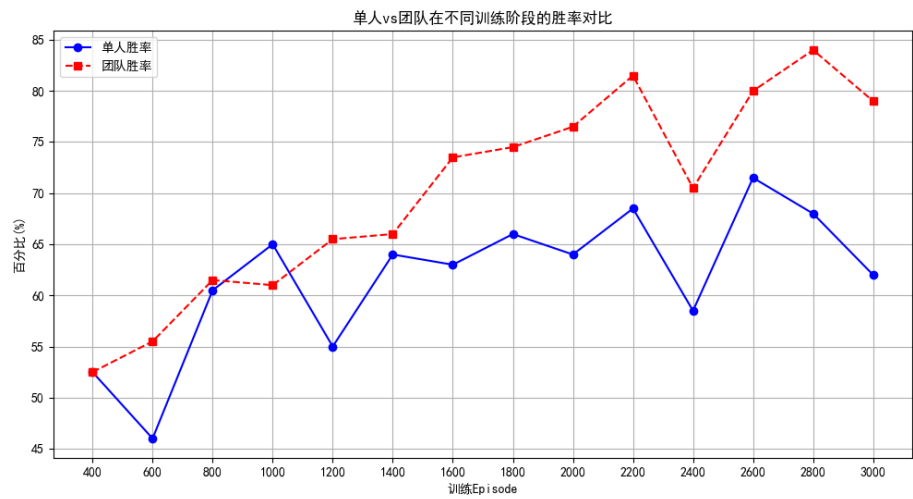


图 3 单人及团队测试胜率折线图

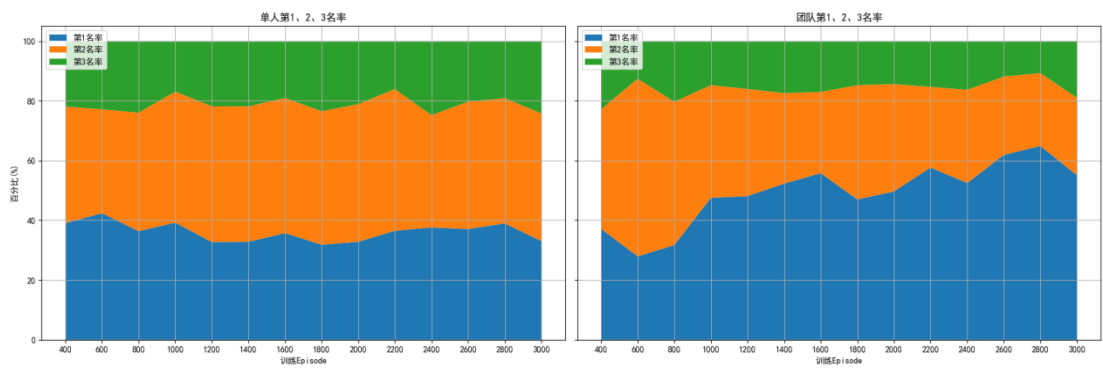


图 4 单人及团队测试胜场排名情况面积图

在项目后期，我们将 Actor 网络使用 Transformer，得到更高的胜率、更快的收敛速度，单人及团队测试胜率折线图见下方图。

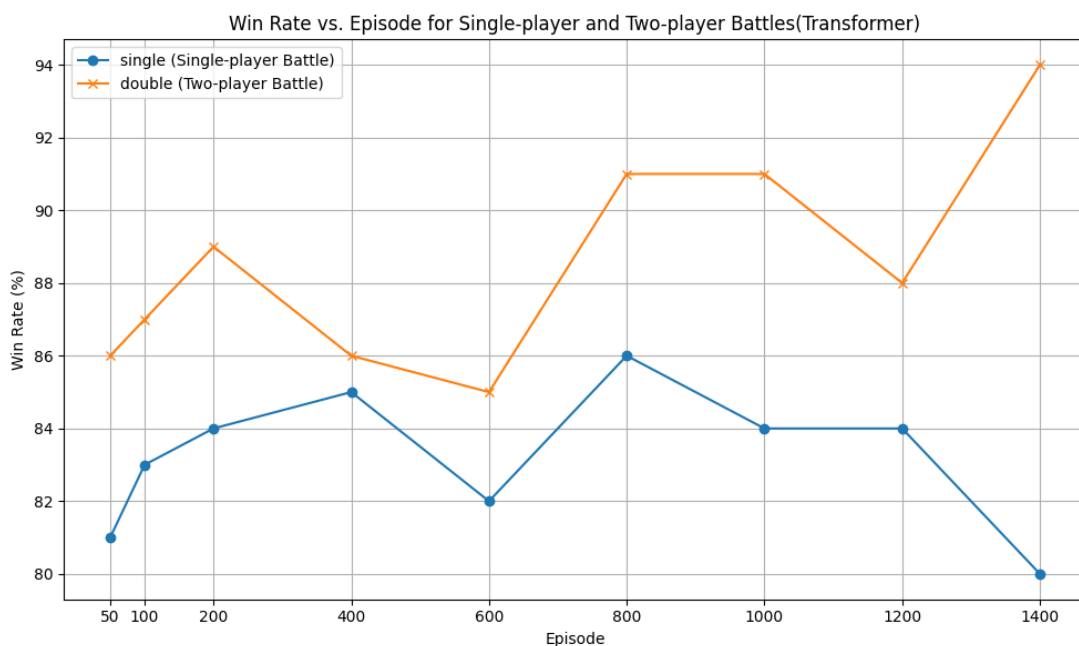


图 5Transformer 单人及团队测试胜率折线图

4.5. 训练效果总结

在前 2200Episode 中模型的综合能力稳定上升，之后边际效应出现，可以认为是受目前模型设置、参数量、奖励函数以及训练环境的限制下已达到较优表现，故停止 3000Episode 后的训练。

我们在 400-3000Episode 的模型中取了一系列模型并命名，作为不同水平的 AI 供玩家选择。

经过测试，表现最好的模型在团队测试中的胜率能够达到 80%以上，且胜场中取得 3 分（即一二名）的比例达到 50%以上。可以认为已经具备一定的掇蛋游戏水平及团队合作意识，但与人类水平的比较仍待验证。

此外，我们采用实战测试了少量对局，从 AI 建议及其概率得到模型初步具备团队合作意识、抢先手意识等掇蛋高阶技巧意识，达到预期效果。

5. 总结与展望

5.1. 工作总结

本项目围绕“掼蛋”这一复杂的四人协作式扑克牌游戏，成功设计并实现了一套基于深度强化学习的 AI 博弈系统。主要工作成果总结如下：

环境搭建与规则实现：我们从零开始构建了一个功能完备的掼蛋虚拟仿真环境。该环境精确地实现了发牌、出牌合法性判断、牌力比较、轮次流转和“接风”等核心游戏机制，为强化学习智能体的训练与测试提供了稳定可靠的平台。

算法选型与模型设计：针对掼蛋不完全信息和高维决策空间的特点，我们采用了主流且高效的演员-评论家（Actor-Critic）算法框架。在模型设计上，我们创新地将游戏状态编码为 3049 维的特征向量，并把所有可能的出牌组合归纳为 375 种结构化动作。这种设计有效平衡了信息完整性与模型学习效率，是本项目能够成功训练的关键。

训练与效果验证：通过自对弈的训练范式，我们的 AI 智能体在与随机策略机器人的对抗中不断迭代优化。我们系统性地测试了不同训练阶段模型的性能，结果显示，AI 的胜率随着训练的进行稳步提升。特别是引入 Transformer 作为策略网络后，模型的性能得到显著增强，团队模式胜率超过 80%，证明其学习到了有效的出牌策略和初步的团队协作能力。从实战效果来看，AI 已经能够做出如主动抢夺出牌权、打出优势牌型压制对手等高级操作。

总而言之，本项目完整地走通了利用深度强化学习解决复杂棋牌博弈问题的技术路径，最终成果达到了预期的研究目标，证明了该方法在掼蛋 AI 领域的应用潜力。

5.2. 问题与不足

尽管本项目取得了阶段性成功，但仍存在一些局限性和待改进之处：

基线对手强度有限：目前 AI 的训练和测试均是“与随机出牌的机器人”进行的。虽然随机策略可以验证模型的基本学习能力，但它是一个较弱的基线，无法真正衡量 AI 在面对有策略的对手（如人类玩家或其他高级 AI）时的表现。

游戏规则的简化：为了降低初期训练的复杂度，当前环境省略了“逢人配”（万能牌）和“贡牌”等掇蛋的特色规则。这些规则极大地影响了游戏的策略深度，模型的缺席意味着其策略与真实对局存在差距。

协作策略的深度：尽管测试结果显示 AI 具备一定的团队意识，但其协作行为是基于最大化团队共同奖励而间接学到的。模型并未显式地对队友的意图进行建模和推理，缺乏更深层次、更主动的协作策略。

模型可解释性不强：与所有深度学习模型类似，我们的 AI 决策过程是一个“黑箱”。我们很难精确地解释 AI 在某一特定局势下做出某个决策的具体原因，这为策略的分析和优化带来了挑战。

5.3. 未来展望

针对上述不足，我们规划了以下几个未来可能的研究方向，以期构建更强大的掇蛋 AI：

增强训练机制：引入历史版本自对弈（Self-Play Against Previous Versions）机制，让 AI 在与过去更强的自己对战中持续进步，摆脱对随机基线的依赖。此外，可以尝试结合模仿学习，利用高质量的人类对局数据对模型进行预训练，使其更快地掌握基本策略和人类牌感。

引入更高级的 AI 算法：探索多智能体强化学习（MARL）算法，如 MADDPG 等，让每个智能体都能显式地对其他玩家（包括队友和对手）的策略进行建模。这有望催生出更复杂、更精妙的团队协作与对抗策略。

完善游戏环境与模型：将“逢人配”、“贡牌”等完整规则加入到环境中，

向完全真实的掼蛋博弈发起挑战。同时，可以研究如图神经网络（GNN）等更先进的模型结构，以更自然地表达扑克牌之间的组合关系。

构建人机交互系统：开发一个用户友好的图形界面，让普通玩家可以与我们训练的 AI 进行对战。这不仅能提供最直观的性能评测，也能收集宝贵的人机对战数据，用于模型的进一步迭代优化，最终实现一个能与人类玩家一较高下的高水平掼蛋 AI。

参考文献

- [1] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6382 – 6393.
- [2] Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Qian, C., Chan, C., Qin, Y., Lu, Y., Xie, R., Liu, Z., Sun, M., & Zhou, J. (2023). AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors in Agents. *ArXiv, abs/2308.10848*.
- [3] Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- [4] Silver, D., et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), 354-359.
- [5] FIPA. (2002). FIPA Agent Communication Language Specification. Geneva: Foundation for Intelligent Physical Agents.
- [6] Dong, Yihong et al. “Self-collaboration Code Generation via ChatGPT.” *ACM*

- Transactions on Software Engineering and Methodology (2023): n. pag.
- [7] Guo, Taicheng et al. “Large Language Model based Multi-Agents: A Survey of Progress and Challenges. ” International Joint Conference on Artificial Intelligence (2024).
 - [8] Yim, Yauwai et al. “Evaluating and Enhancing LLMs Agent based on Theory of Mind in Guandan: A Multi-Player Cooperative Game under Imperfect Information.” ArXiv abs/2408.02559 (2024): n. pag.
 - [9] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Playing Atari with Deep Reinforcement Learning[C] // Proceedings of Workshops at the 26th Neural Information Processing Systems 2013. Lake Tahoe, USA: NIPS Press, 2013: 201-220.
 - [10] VAN HASSELT H, GUEZ A, SILVER D. Deep Reinforcement Learning with Double Q-Learning[C] // AAAI. Arizona: AAAAI Press, 2016: 5.
 - [11] SCHAU T, QUAN J, ANTONOGLOU I, et al. Prioritized experience replay[C] // Proceedings of the 4th International Conference on Learning Representations. San Juan, Puerto Rico: ICLR Press, 2016: 322-355.
 - [12] OSBAND I, VAN ROY B, WEN Z. Generalization and exploration via randomized value functions[J]. Proceedings of the 33rd International Conference on Machine Learning, 2014, 48(1): 2377-2386.
 - [13] MUNOS R, STEPLETON T, HARUTYUNYAN A, et al. Safe and efficient off-policy reinforcement learning[C] // Advances in Neural Information Processing Systems. Spain: NIPS Press, 2016: 1054-1062.
 - [14] FRANÇOIS-LAVET V, FONTENEAU R, ERNST D. How to discount deep reinforcement learning: Towards new dynamic strategies[C] // Proceedings of the Workshops at the Advances in Neural Information Processing Systems. Montreal, Canada: NIPS Press, 2015: 1107-1160.

- [15] LILLICRAP T P, HUNT J J, PRITZEL A, et al. Continuous control with deep reinforcement learning: U.S. Patent Application 15/217,758[P]. 2017-01-26.
- [16] MNIH V, BADIA A P, MIRZA M, et al. Asynchronous methods for deep reinforcement learning[C] // International Conference on Machine Learning. New York City: ICML Press, 2016: 1928-1937.
- [17] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms[J/OL].arXiv:1707.06347.
- [18] HEESS N, SRIRAM S, LEMMON J, et al. Emergence of locomotion behaviors in rich environments[J/OL].arXiv:1707.02286.

附录

表格 5 完整的动作空间

type	points	logic_point	id	suit	a_as
None	[]	0	0	NULL	NULL
single	[2]	2	1	NULL	NULL
single	[3]	3	2	NULL	NULL
single	[4]	4	3	NULL	NULL
single	[5]	5	4	NULL	NULL
single	[6]	6	5	NULL	NULL
single	[7]	7	6	NULL	NULL
single	[8]	8	7	NULL	NULL
single	[9]	9	8	NULL	NULL
single	[10]	10	9	NULL	NULL
single	[11]	11	10	NULL	NULL
single	[12]	12	11	NULL	NULL
single	[13]	13	12	NULL	NULL
single	[14]	14	13	NULL	NULL
single	[15]	15	14	非红桃	NULL

type	points	logic_point	id	suit	a_as
single	[15]	15	15	红桃	NULL
single	[16]	16	16	NULL	NULL
single	[17]	17	17	NULL	NULL
pair	[2, 2]	2	18	NULL	NULL
pair	[3, 3]	3	19	NULL	NULL
pair	[4, 4]	4	20	NULL	NULL
pair	[5, 5]	5	21	NULL	NULL
pair	[6, 6]	6	22	NULL	NULL
pair	[7, 7]	7	23	NULL	NULL
pair	[8, 8]	8	24	NULL	NULL
pair	[9, 9]	9	25	NULL	NULL
pair	[10, 10]	10	26	NULL	NULL
pair	[11, 11]	11	27	NULL	NULL
pair	[12, 12]	12	28	NULL	NULL
pair	[13, 13]	13	29	NULL	NULL
pair	[14, 14]	14	30	NULL	NULL
pair	[15, 15]	15	31	非红桃	NULL
pair	[15, 15]	15	32	红桃	NULL
pair	[16, 16]	16	33	NULL	NULL
pair	[17, 17]	17	34	NULL	NULL
triple	[2, 2, 2]	2	35	NULL	NULL
triple	[3, 3, 3]	3	36	NULL	NULL
triple	[4, 4, 4]	4	37	NULL	NULL
triple	[5, 5, 5]	5	38	NULL	NULL
triple	[6, 6, 6]	6	39	NULL	NULL
triple	[7, 7, 7]	7	40	NULL	NULL
triple	[8, 8, 8]	8	41	NULL	NULL
triple	[9, 9, 9]	9	42	NULL	NULL
triple	[10, 10, 10]	10	43	NULL	NULL
triple	[11, 11, 11]	11	44	NULL	NULL
triple	[12, 12, 12]	12	45	NULL	NULL
triple	[13, 13, 13]	13	46	NULL	NULL
triple	[14, 14, 14]	14	47	NULL	NULL
triple	[15, 15, 15]	15	48	NULL	NULL
4_bomb	[2, 2, 2, 2]	2	49	NULL	NULL
4_bomb	[3, 3, 3, 3]	3	50	NULL	NULL
4_bomb	[4, 4, 4, 4]	4	51	NULL	NULL
4_bomb	[5, 5, 5, 5]	5	52	NULL	NULL
4_bomb	[6, 6, 6, 6]	6	53	NULL	NULL
4_bomb	[7, 7, 7, 7]	7	54	NULL	NULL
4_bomb	[8, 8, 8, 8]	8	55	NULL	NULL
4_bomb	[9, 9, 9, 9]	9	56	NULL	NULL

type	points	logic_point	id	suit	a_as
4_bomb	[10, 10, 10, 10]	10	57	NULL	NULL
4_bomb	[11, 11, 11, 11]	11	58	NULL	NULL
4_bomb	[12, 12, 12, 12]	12	59	NULL	NULL
4_bomb	[13, 13, 13, 13]	13	60	NULL	NULL
4_bomb	[14, 14, 14, 14]	14	61	NULL	NULL
4_bomb	[15, 15, 15, 15]	15	62	NULL	NULL
5_bomb	[2, 2, 2, 2, 2]	2	63	NULL	NULL
5_bomb	[3, 3, 3, 3, 3]	3	64	NULL	NULL
5_bomb	[4, 4, 4, 4, 4]	4	65	NULL	NULL
5_bomb	[5, 5, 5, 5, 5]	5	66	NULL	NULL
5_bomb	[6, 6, 6, 6, 6]	6	67	NULL	NULL
5_bomb	[7, 7, 7, 7, 7]	7	68	NULL	NULL
5_bomb	[8, 8, 8, 8, 8]	8	69	NULL	NULL
5_bomb	[9, 9, 9, 9, 9]	9	70	NULL	NULL
5_bomb	[10, 10, 10, 10, 10]	10	71	NULL	NULL
5_bomb	[11, 11, 11, 11, 11]	11	72	NULL	NULL
5_bomb	[12, 12, 12, 12, 12]	12	73	NULL	NULL
5_bomb	[13, 13, 13, 13, 13]	13	74	NULL	NULL
5_bomb	[14, 14, 14, 14, 14]	14	75	NULL	NULL
5_bomb	[15, 15, 15, 15, 15]	15	76	NULL	NULL
6_bomb	[2, 2, 2, 2, 2, 2]	2	77	NULL	NULL
6_bomb	[3, 3, 3, 3, 3, 3]	3	78	NULL	NULL
6_bomb	[4, 4, 4, 4, 4, 4]	4	79	NULL	NULL
6_bomb	[5, 5, 5, 5, 5, 5]	5	80	NULL	NULL
6_bomb	[6, 6, 6, 6, 6, 6]	6	81	NULL	NULL
6_bomb	[7, 7, 7, 7, 7, 7]	7	82	NULL	NULL
6_bomb	[8, 8, 8, 8, 8, 8]	8	83	NULL	NULL
6_bomb	[9, 9, 9, 9, 9, 9]	9	84	NULL	NULL
6_bomb	[10, 10, 10, 10, 10, 10]	10	85	NULL	NULL
6_bomb	[11, 11, 11, 11, 11, 11]	11	86	NULL	NULL
6_bomb	[12, 12, 12, 12, 12, 12]	12	87	NULL	NULL
6_bomb	[13, 13, 13, 13, 13, 13]	13	88	NULL	NULL
6_bomb	[14, 14, 14, 14, 14, 14]	14	89	NULL	NULL
6_bomb	[15, 15, 15, 15, 15, 15]	15	90	NULL	NULL
7_bomb	[2, 2, 2, 2, 2, 2, 2]	2	91	NULL	NULL
7_bomb	[3, 3, 3, 3, 3, 3, 3]	3	92	NULL	NULL
7_bomb	[4, 4, 4, 4, 4, 4, 4]	4	93	NULL	NULL
7_bomb	[5, 5, 5, 5, 5, 5, 5]	5	94	NULL	NULL
7_bomb	[6, 6, 6, 6, 6, 6, 6]	6	95	NULL	NULL
7_bomb	[7, 7, 7, 7, 7, 7, 7]	7	96	NULL	NULL
7_bomb	[8, 8, 8, 8, 8, 8, 8]	8	97	NULL	NULL
7_bomb	[9, 9, 9, 9, 9, 9, 9]	9	98	NULL	NULL

type	points	logic_point	id	suit	a_as
7_bomb	[10, 10, 10, 10, 10, 10, 10]	10	99	NULL	NULL
7_bomb	[11, 11, 11, 11, 11, 11, 11]	11	100	NULL	NULL
7_bomb	[12, 12, 12, 12, 12, 12, 12]	12	101	NULL	NULL
7_bomb	[13, 13, 13, 13, 13, 13, 13]	13	102	NULL	NULL
7_bomb	[14, 14, 14, 14, 14, 14, 14]	14	103	NULL	NULL
7_bomb	[15, 15, 15, 15, 15, 15, 15]	15	104	NULL	NULL
8_bomb	[2, 2, 2, 2, 2, 2, 2, 2]	2	105	NULL	NULL
8_bomb	[3, 3, 3, 3, 3, 3, 3, 3]	3	106	NULL	NULL
8_bomb	[4, 4, 4, 4, 4, 4, 4, 4]	4	107	NULL	NULL
8_bomb	[5, 5, 5, 5, 5, 5, 5, 5]	5	108	NULL	NULL
8_bomb	[6, 6, 6, 6, 6, 6, 6, 6]	6	109	NULL	NULL
8_bomb	[7, 7, 7, 7, 7, 7, 7, 7]	7	110	NULL	NULL
8_bomb	[8, 8, 8, 8, 8, 8, 8, 8]	8	111	NULL	NULL
8_bomb	[9, 9, 9, 9, 9, 9, 9, 9]	9	112	NULL	NULL
8_bomb	[10, 10, 10, 10, 10, 10, 10, 10]	10	113	NULL	NULL
8_bomb	[11, 11, 11, 11, 11, 11, 11, 11]	11	114	NULL	NULL
8_bomb	[12, 12, 12, 12, 12, 12, 12, 12]	12	115	NULL	NULL
8_bomb	[13, 13, 13, 13, 13, 13, 13, 13]	13	116	NULL	NULL
8_bomb	[14, 14, 14, 14, 14, 14, 14, 14]	14	117	NULL	NULL
8_bomb	[15, 15, 15, 15, 15, 15, 15, 15]	15	118	NULL	NULL
joker_bomb	[16, 16, 17, 17]	1	119	NULL	NULL
three_with_pair	[2, 2, 2, 3, 3]	2	120	NULL	NULL
three_with_pair	[2, 2, 2, 4, 4]	2	121	NULL	NULL
three_with_pair	[2, 2, 2, 5, 5]	2	122	NULL	NULL
three_with_pair	[2, 2, 2, 6, 6]	2	123	NULL	NULL
three_with_pair	[2, 2, 2, 7, 7]	2	124	NULL	NULL
three_with_pair	[2, 2, 2, 8, 8]	2	125	NULL	NULL
three_with_pair	[2, 2, 2, 9, 9]	2	126	NULL	NULL
three_with_pair	[2, 2, 2, 10, 10]	2	127	NULL	NULL
three_with_pair	[2, 2, 2, 11, 11]	2	128	NULL	NULL
three_with_pair	[2, 2, 2, 12, 12]	2	129	NULL	NULL
three_with_pair	[2, 2, 2, 13, 13]	2	130	NULL	NULL
three_with_pair	[2, 2, 2, 14, 14]	2	131	NULL	NULL
three_with_pair	[2, 2, 2, 15, 15]	2	132	NULL	NULL
three_with_pair	[2, 2, 2, 16, 16]	2	133	NULL	NULL
three_with_pair	[2, 2, 2, 17, 17]	2	134	NULL	NULL
three_with_pair	[3, 3, 3, 2, 2]	3	135	NULL	NULL
three_with_pair	[3, 3, 3, 4, 4]	3	136	NULL	NULL
three_with_pair	[3, 3, 3, 5, 5]	3	137	NULL	NULL
three_with_pair	[3, 3, 3, 6, 6]	3	138	NULL	NULL
three_with_pair	[3, 3, 3, 7, 7]	3	139	NULL	NULL
three_with_pair	[3, 3, 3, 8, 8]	3	140	NULL	NULL

type	points	logic_point	id	suit	a_as
three_with_pair	[3, 3, 3, 9, 9]	3	141	NULL	NULL
three_with_pair	[3, 3, 3, 10, 10]	3	142	NULL	NULL
three_with_pair	[3, 3, 3, 11, 11]	3	143	NULL	NULL
three_with_pair	[3, 3, 3, 12, 12]	3	144	NULL	NULL
three_with_pair	[3, 3, 3, 13, 13]	3	145	NULL	NULL
three_with_pair	[3, 3, 3, 14, 14]	3	146	NULL	NULL
three_with_pair	[3, 3, 3, 15, 15]	3	147	NULL	NULL
three_with_pair	[3, 3, 3, 16, 16]	3	148	NULL	NULL
three_with_pair	[3, 3, 3, 17, 17]	3	149	NULL	NULL
three_with_pair	[4, 4, 4, 2, 2]	4	150	NULL	NULL
three_with_pair	[4, 4, 4, 3, 3]	4	151	NULL	NULL
three_with_pair	[4, 4, 4, 5, 5]	4	152	NULL	NULL
three_with_pair	[4, 4, 4, 6, 6]	4	153	NULL	NULL
three_with_pair	[4, 4, 4, 7, 7]	4	154	NULL	NULL
three_with_pair	[4, 4, 4, 8, 8]	4	155	NULL	NULL
three_with_pair	[4, 4, 4, 9, 9]	4	156	NULL	NULL
three_with_pair	[4, 4, 4, 10, 10]	4	157	NULL	NULL
three_with_pair	[4, 4, 4, 11, 11]	4	158	NULL	NULL
three_with_pair	[4, 4, 4, 12, 12]	4	159	NULL	NULL
three_with_pair	[4, 4, 4, 13, 13]	4	160	NULL	NULL
three_with_pair	[4, 4, 4, 14, 14]	4	161	NULL	NULL
three_with_pair	[4, 4, 4, 15, 15]	4	162	NULL	NULL
three_with_pair	[4, 4, 4, 16, 16]	4	163	NULL	NULL
three_with_pair	[4, 4, 4, 17, 17]	4	164	NULL	NULL
three_with_pair	[5, 5, 5, 2, 2]	5	165	NULL	NULL
three_with_pair	[5, 5, 5, 3, 3]	5	166	NULL	NULL
three_with_pair	[5, 5, 5, 4, 4]	5	167	NULL	NULL
three_with_pair	[5, 5, 5, 6, 6]	5	168	NULL	NULL
three_with_pair	[5, 5, 5, 7, 7]	5	169	NULL	NULL
three_with_pair	[5, 5, 5, 8, 8]	5	170	NULL	NULL
three_with_pair	[5, 5, 5, 9, 9]	5	171	NULL	NULL
three_with_pair	[5, 5, 5, 10, 10]	5	172	NULL	NULL
three_with_pair	[5, 5, 5, 11, 11]	5	173	NULL	NULL
three_with_pair	[5, 5, 5, 12, 12]	5	174	NULL	NULL
three_with_pair	[5, 5, 5, 13, 13]	5	175	NULL	NULL
three_with_pair	[5, 5, 5, 14, 14]	5	176	NULL	NULL
three_with_pair	[5, 5, 5, 15, 15]	5	177	NULL	NULL
three_with_pair	[5, 5, 5, 16, 16]	5	178	NULL	NULL
three_with_pair	[5, 5, 5, 17, 17]	5	179	NULL	NULL
three_with_pair	[6, 6, 6, 2, 2]	6	180	NULL	NULL
three_with_pair	[6, 6, 6, 3, 3]	6	181	NULL	NULL
three_with_pair	[6, 6, 6, 4, 4]	6	182	NULL	NULL

type	points	logic_point	id	suit	a_as
three_with_pair	[6, 6, 6, 5, 5]	6	183	NULL	NULL
three_with_pair	[6, 6, 6, 7, 7]	6	184	NULL	NULL
three_with_pair	[6, 6, 6, 8, 8]	6	185	NULL	NULL
three_with_pair	[6, 6, 6, 9, 9]	6	186	NULL	NULL
three_with_pair	[6, 6, 6, 10, 10]	6	187	NULL	NULL
three_with_pair	[6, 6, 6, 11, 11]	6	188	NULL	NULL
three_with_pair	[6, 6, 6, 12, 12]	6	189	NULL	NULL
three_with_pair	[6, 6, 6, 13, 13]	6	190	NULL	NULL
three_with_pair	[6, 6, 6, 14, 14]	6	191	NULL	NULL
three_with_pair	[6, 6, 6, 15, 15]	6	192	NULL	NULL
three_with_pair	[6, 6, 6, 16, 16]	6	193	NULL	NULL
three_with_pair	[6, 6, 6, 17, 17]	6	194	NULL	NULL
three_with_pair	[7, 7, 7, 2, 2]	7	195	NULL	NULL
three_with_pair	[7, 7, 7, 3, 3]	7	196	NULL	NULL
three_with_pair	[7, 7, 7, 4, 4]	7	197	NULL	NULL
three_with_pair	[7, 7, 7, 5, 5]	7	198	NULL	NULL
three_with_pair	[7, 7, 7, 6, 6]	7	199	NULL	NULL
three_with_pair	[7, 7, 7, 8, 8]	7	200	NULL	NULL
three_with_pair	[7, 7, 7, 9, 9]	7	201	NULL	NULL
three_with_pair	[7, 7, 7, 10, 10]	7	202	NULL	NULL
three_with_pair	[7, 7, 7, 11, 11]	7	203	NULL	NULL
three_with_pair	[7, 7, 7, 12, 12]	7	204	NULL	NULL
three_with_pair	[7, 7, 7, 13, 13]	7	205	NULL	NULL
three_with_pair	[7, 7, 7, 14, 14]	7	206	NULL	NULL
three_with_pair	[7, 7, 7, 15, 15]	7	207	NULL	NULL
three_with_pair	[7, 7, 7, 16, 16]	7	208	NULL	NULL
three_with_pair	[7, 7, 7, 17, 17]	7	209	NULL	NULL
three_with_pair	[8, 8, 8, 2, 2]	8	210	NULL	NULL
three_with_pair	[8, 8, 8, 3, 3]	8	211	NULL	NULL
three_with_pair	[8, 8, 8, 4, 4]	8	212	NULL	NULL
three_with_pair	[8, 8, 8, 5, 5]	8	213	NULL	NULL
three_with_pair	[8, 8, 8, 6, 6]	8	214	NULL	NULL
three_with_pair	[8, 8, 8, 7, 7]	8	215	NULL	NULL
three_with_pair	[8, 8, 8, 9, 9]	8	216	NULL	NULL
three_with_pair	[8, 8, 8, 10, 10]	8	217	NULL	NULL
three_with_pair	[8, 8, 8, 11, 11]	8	218	NULL	NULL
three_with_pair	[8, 8, 8, 12, 12]	8	219	NULL	NULL
three_with_pair	[8, 8, 8, 13, 13]	8	220	NULL	NULL
three_with_pair	[8, 8, 8, 14, 14]	8	221	NULL	NULL
three_with_pair	[8, 8, 8, 15, 15]	8	222	NULL	NULL
three_with_pair	[8, 8, 8, 16, 16]	8	223	NULL	NULL
three_with_pair	[8, 8, 8, 17, 17]	8	224	NULL	NULL

type	points	logic_point	id	suit	a_as
three_with_pair	[9, 9, 9, 2, 2]	9	225	NULL	NULL
three_with_pair	[9, 9, 9, 3, 3]	9	226	NULL	NULL
three_with_pair	[9, 9, 9, 4, 4]	9	227	NULL	NULL
three_with_pair	[9, 9, 9, 5, 5]	9	228	NULL	NULL
three_with_pair	[9, 9, 9, 6, 6]	9	229	NULL	NULL
three_with_pair	[9, 9, 9, 7, 7]	9	230	NULL	NULL
three_with_pair	[9, 9, 9, 8, 8]	9	231	NULL	NULL
three_with_pair	[9, 9, 9, 10, 10]	9	232	NULL	NULL
three_with_pair	[9, 9, 9, 11, 11]	9	233	NULL	NULL
three_with_pair	[9, 9, 9, 12, 12]	9	234	NULL	NULL
three_with_pair	[9, 9, 9, 13, 13]	9	235	NULL	NULL
three_with_pair	[9, 9, 9, 14, 14]	9	236	NULL	NULL
three_with_pair	[9, 9, 9, 15, 15]	9	237	NULL	NULL
three_with_pair	[9, 9, 9, 16, 16]	9	238	NULL	NULL
three_with_pair	[9, 9, 9, 17, 17]	9	239	NULL	NULL
three_with_pair	[10, 10, 10, 2, 2]	10	240	NULL	NULL
three_with_pair	[10, 10, 10, 3, 3]	10	241	NULL	NULL
three_with_pair	[10, 10, 10, 4, 4]	10	242	NULL	NULL
three_with_pair	[10, 10, 10, 5, 5]	10	243	NULL	NULL
three_with_pair	[10, 10, 10, 6, 6]	10	244	NULL	NULL
three_with_pair	[10, 10, 10, 7, 7]	10	245	NULL	NULL
three_with_pair	[10, 10, 10, 8, 8]	10	246	NULL	NULL
three_with_pair	[10, 10, 10, 9, 9]	10	247	NULL	NULL
three_with_pair	[10, 10, 10, 11, 11]	10	248	NULL	NULL
three_with_pair	[10, 10, 10, 12, 12]	10	249	NULL	NULL
three_with_pair	[10, 10, 10, 13, 13]	10	250	NULL	NULL
three_with_pair	[10, 10, 10, 14, 14]	10	251	NULL	NULL
three_with_pair	[10, 10, 10, 15, 15]	10	252	NULL	NULL
three_with_pair	[10, 10, 10, 16, 16]	10	253	NULL	NULL
three_with_pair	[10, 10, 10, 17, 17]	10	254	NULL	NULL
three_with_pair	[11, 11, 11, 2, 2]	11	255	NULL	NULL
three_with_pair	[11, 11, 11, 3, 3]	11	256	NULL	NULL
three_with_pair	[11, 11, 11, 4, 4]	11	257	NULL	NULL
three_with_pair	[11, 11, 11, 5, 5]	11	258	NULL	NULL
three_with_pair	[11, 11, 11, 6, 6]	11	259	NULL	NULL
three_with_pair	[11, 11, 11, 7, 7]	11	260	NULL	NULL
three_with_pair	[11, 11, 11, 8, 8]	11	261	NULL	NULL
three_with_pair	[11, 11, 11, 9, 9]	11	262	NULL	NULL
three_with_pair	[11, 11, 11, 10, 10]	11	263	NULL	NULL
three_with_pair	[11, 11, 11, 12, 12]	11	264	NULL	NULL
three_with_pair	[11, 11, 11, 13, 13]	11	265	NULL	NULL
three_with_pair	[11, 11, 11, 14, 14]	11	266	NULL	NULL

type	points	logic_point	id	suit	a_as
three_with_pair	[11, 11, 11, 15, 15]	11	267	NULL	NULL
three_with_pair	[11, 11, 11, 16, 16]	11	268	NULL	NULL
three_with_pair	[11, 11, 11, 17, 17]	11	269	NULL	NULL
three_with_pair	[12, 12, 12, 2, 2]	12	270	NULL	NULL
three_with_pair	[12, 12, 12, 3, 3]	12	271	NULL	NULL
three_with_pair	[12, 12, 12, 4, 4]	12	272	NULL	NULL
three_with_pair	[12, 12, 12, 5, 5]	12	273	NULL	NULL
three_with_pair	[12, 12, 12, 6, 6]	12	274	NULL	NULL
three_with_pair	[12, 12, 12, 7, 7]	12	275	NULL	NULL
three_with_pair	[12, 12, 12, 8, 8]	12	276	NULL	NULL
three_with_pair	[12, 12, 12, 9, 9]	12	277	NULL	NULL
three_with_pair	[12, 12, 12, 10, 10]	12	278	NULL	NULL
three_with_pair	[12, 12, 12, 11, 11]	12	279	NULL	NULL
three_with_pair	[12, 12, 12, 13, 13]	12	280	NULL	NULL
three_with_pair	[12, 12, 12, 14, 14]	12	281	NULL	NULL
three_with_pair	[12, 12, 12, 15, 15]	12	282	NULL	NULL
three_with_pair	[12, 12, 12, 16, 16]	12	283	NULL	NULL
three_with_pair	[12, 12, 12, 17, 17]	12	284	NULL	NULL
three_with_pair	[13, 13, 13, 2, 2]	13	285	NULL	NULL
three_with_pair	[13, 13, 13, 3, 3]	13	286	NULL	NULL
three_with_pair	[13, 13, 13, 4, 4]	13	287	NULL	NULL
three_with_pair	[13, 13, 13, 5, 5]	13	288	NULL	NULL
three_with_pair	[13, 13, 13, 6, 6]	13	289	NULL	NULL
three_with_pair	[13, 13, 13, 7, 7]	13	290	NULL	NULL
three_with_pair	[13, 13, 13, 8, 8]	13	291	NULL	NULL
three_with_pair	[13, 13, 13, 9, 9]	13	292	NULL	NULL
three_with_pair	[13, 13, 13, 10, 10]	13	293	NULL	NULL
three_with_pair	[13, 13, 13, 11, 11]	13	294	NULL	NULL
three_with_pair	[13, 13, 13, 12, 12]	13	295	NULL	NULL
three_with_pair	[13, 13, 13, 14, 14]	13	296	NULL	NULL
three_with_pair	[13, 13, 13, 15, 15]	13	297	NULL	NULL
three_with_pair	[13, 13, 13, 16, 16]	13	298	NULL	NULL
three_with_pair	[13, 13, 13, 17, 17]	13	299	NULL	NULL
three_with_pair	[14, 14, 14, 2, 2]	14	300	NULL	NULL
three_with_pair	[14, 14, 14, 3, 3]	14	301	NULL	NULL
three_with_pair	[14, 14, 14, 4, 4]	14	302	NULL	NULL
three_with_pair	[14, 14, 14, 5, 5]	14	303	NULL	NULL
three_with_pair	[14, 14, 14, 6, 6]	14	304	NULL	NULL
three_with_pair	[14, 14, 14, 7, 7]	14	305	NULL	NULL
three_with_pair	[14, 14, 14, 8, 8]	14	306	NULL	NULL
three_with_pair	[14, 14, 14, 9, 9]	14	307	NULL	NULL
three_with_pair	[14, 14, 14, 10, 10]	14	308	NULL	NULL

type	points	logic_point	id	suit	a_as
three_with_pair	[14, 14, 14, 11, 11]	14	309	NULL	NULL
three_with_pair	[14, 14, 14, 12, 12]	14	310	NULL	NULL
three_with_pair	[14, 14, 14, 13, 13]	14	311	NULL	NULL
three_with_pair	[14, 14, 14, 15, 15]	14	312	NULL	NULL
three_with_pair	[14, 14, 14, 16, 16]	14	313	NULL	NULL
three_with_pair	[14, 14, 14, 17, 17]	14	314	NULL	NULL
three_with_pair	[15, 15, 15, 2, 2]	15	315	NULL	NULL
three_with_pair	[15, 15, 15, 3, 3]	15	316	NULL	NULL
three_with_pair	[15, 15, 15, 4, 4]	15	317	NULL	NULL
three_with_pair	[15, 15, 15, 5, 5]	15	318	NULL	NULL
three_with_pair	[15, 15, 15, 6, 6]	15	319	NULL	NULL
three_with_pair	[15, 15, 15, 7, 7]	15	320	NULL	NULL
three_with_pair	[15, 15, 15, 8, 8]	15	321	NULL	NULL
three_with_pair	[15, 15, 15, 9, 9]	15	322	NULL	NULL
three_with_pair	[15, 15, 15, 10, 10]	15	323	NULL	NULL
three_with_pair	[15, 15, 15, 11, 11]	15	324	NULL	NULL
three_with_pair	[15, 15, 15, 12, 12]	15	325	NULL	NULL
three_with_pair	[15, 15, 15, 13, 13]	15	326	NULL	NULL
three_with_pair	[15, 15, 15, 14, 14]	15	327	NULL	NULL
three_with_pair	[15, 15, 15, 16, 16]	15	328	NULL	NULL
three_with_pair	[15, 15, 15, 17, 17]	15	329	NULL	NULL
straight	[2, 3, 4, 5, 14]	1	330	NULL	low
straight	[2, 3, 4, 5, 6]	2	331	NULL	high
straight	[3, 4, 5, 6, 7]	3	332	NULL	high
straight	[4, 5, 6, 7, 8]	4	333	NULL	high
straight	[5, 6, 7, 8, 9]	5	334	NULL	high
straight	[6, 7, 8, 9, 10]	6	335	NULL	high
straight	[7, 8, 9, 10, 11]	7	336	NULL	high
straight	[8, 9, 10, 11, 12]	8	337	NULL	high
straight	[9, 10, 11, 12, 13]	9	338	NULL	high
straight	[10, 11, 12, 13, 14]	10	339	NULL	high
pair_chain	[2, 2, 3, 3, 14, 14]	1	340	NULL	low
pair_chain	[2, 2, 3, 3, 4, 4]	2	341	NULL	high
pair_chain	[3, 3, 4, 4, 5, 5]	3	342	NULL	high
pair_chain	[4, 4, 5, 5, 6, 6]	4	343	NULL	high
pair_chain	[5, 5, 6, 6, 7, 7]	5	344	NULL	high
pair_chain	[6, 6, 7, 7, 8, 8]	6	345	NULL	high
pair_chain	[7, 7, 8, 8, 9, 9]	7	346	NULL	high
pair_chain	[8, 8, 9, 9, 10, 10]	8	347	NULL	high
pair_chain	[9, 9, 10, 10, 11, 11]	9	348	NULL	high
pair_chain	[10, 10, 11, 11, 12, 12]	10	349	NULL	high
pair_chain	[11, 11, 12, 12, 13, 13]	11	350	NULL	high

type	points	logic_point	id	suit	a_as
pair_chain	[12, 12, 13, 13, 14, 14]	12	351	NULL	high
gangban	[2, 2, 2, 14, 14, 14]	1	352	NULL	low
gangban	[2, 2, 2, 3, 3, 3]	2	353	NULL	high
gangban	[3, 3, 3, 4, 4, 4]	3	354	NULL	high
gangban	[4, 4, 4, 5, 5, 5]	4	355	NULL	high
gangban	[5, 5, 5, 6, 6, 6]	5	356	NULL	high
gangban	[6, 6, 6, 7, 7, 7]	6	357	NULL	high
gangban	[7, 7, 7, 8, 8, 8]	7	358	NULL	high
gangban	[8, 8, 8, 9, 9, 9]	8	359	NULL	high
gangban	[9, 9, 9, 10, 10, 10]	9	360	NULL	high
gangban	[10, 10, 10, 11, 11, 11]	10	361	NULL	high
gangban	[11, 11, 11, 12, 12, 12]	11	362	NULL	high
gangban	[12, 12, 12, 13, 13, 13]	12	363	NULL	high
gangban	[13, 13, 13, 14, 14, 14]	13	364	NULL	high
flush_rocket	[2, 3, 4, 5, 14]	1	365	NULL	low
flush_rocket	[2, 3, 4, 5, 6]	2	366	NULL	high
flush_rocket	[3, 4, 5, 6, 7]	3	367	NULL	high
flush_rocket	[4, 5, 6, 7, 8]	4	368	NULL	high
flush_rocket	[5, 6, 7, 8, 9]	5	369	NULL	high
flush_rocket	[6, 7, 8, 9, 10]	6	370	NULL	high
flush_rocket	[7, 8, 9, 10, 11]	7	371	NULL	high
flush_rocket	[8, 9, 10, 11, 12]	8	372	NULL	high
flush_rocket	[9, 10, 11, 12, 13]	9	373	NULL	high
flush_rocket	[10, 11, 12, 13, 14]	10	374	NULL	high

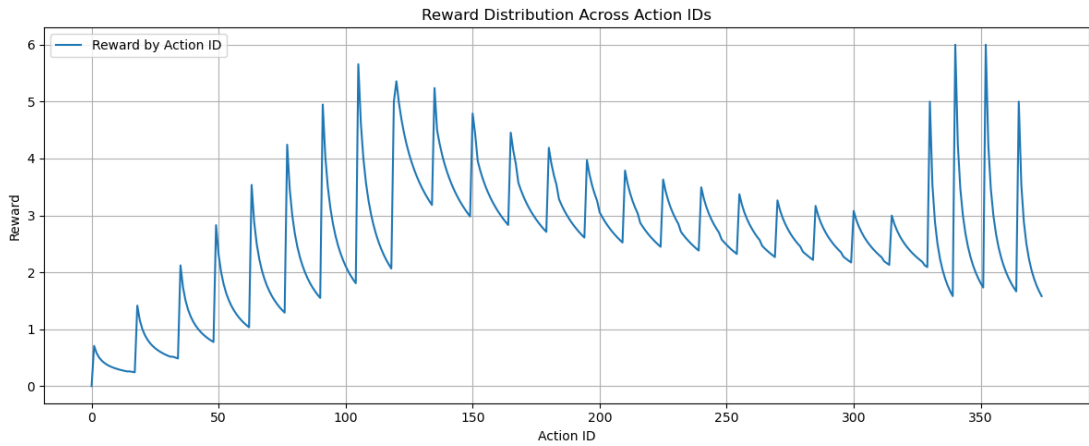


图 6 动作对应的即时奖励